

Family Paws Dog Daycare & Boarding Website

Final Project Documentation

Tanya Artz | CA 1040 Website Development | July 2026

1. Project Title and Overview

Family Paws is a front-end website built for a fictional dog daycare and boarding business. The site gives potential customers clear information about the business, its services and pricing, a photo gallery, and a way to get in touch.

The final website closely follows the original proposal. All four planned pages (Home, Services, Gallery, Contact) were built, along with every core feature described in the proposal: responsive layout, a pricing table, an expandable FAQ, a JavaScript image slideshow, and a contact form. The only notable change from the original plan is that royalty-free stock photography (Unsplash) was used in place of custom photography, since Family Paws is a fictional business with no real facility to photograph. No planned features were cut, and no backend or advanced functionality (booking, accounts, payments) was added, consistent with the original scope decision.

2. Target Audience

The site is designed for dog owners roughly age 25–60 who need daycare, boarding, or grooming services — including working professionals, families, and frequent travelers who need to know a provider is trustworthy before booking.

The final product addresses this audience directly: the Services page states pricing and expectations up front so visitors don’t have to call to get basic answers; the FAQ answers the questions this audience is most likely to have (vaccination requirements, facility tours, how dogs are grouped); the Gallery gives a visual sense of the environment; and the Contact page offers a low-friction way to reach out. The warm, welcoming visual design and simple navigation were chosen to build the kind of trust this audience looks for before leaving their dog with a new provider.

3. Core Features

The table below maps each feature to the technologies used to build it.

Feature	Technology Used	Description
Multi-page site structure	HTML	Four linked pages (Home, Services, Gallery, Contact) sharing a common header, navigation, and footer.
Responsive layout	CSS (Flexbox, media queries)	Service cards use Flexbox; a breakpoint at 700px collapses the layout for mobile and tablet screens.
Mobile navigation menu	JavaScript + CSS	A Menu button toggles a "show" class on the nav element (classList.toggle), revealing links on small screens.
Services & pricing table	HTML + CSS	A styled table lists each service, its description, and starting price.
Expandable FAQ	JavaScript + CSS	Clicking a question toggles the display style of its answer, expanding and collapsing it in place.
Photo gallery slideshow	JavaScript	An array of image objects drives a Previous/Next slideshow; an index variable tracks the current image and

		wraps at either end.
Contact form	HTML + JavaScript	Submitting the form calls <code>preventDefault()</code> , displays a confirmation message, and resets the fields (no real data is sent, per project scope).
Hover effects	CSS	Buttons and navigation links darken or underline on hover to signal interactivity.
Accessibility support	HTML	Descriptive alt text on all images, label/for associations on form fields, and a logical heading structure (<code>h1 > h2 > h3</code>).

4. Design and User Experience Considerations

Visual design

The site uses a warm, neutral color palette — brown (#7B4F35), tan (#F2D6B3), and a soft off-white background (#FFFAF4) — to create a friendly, welcoming feeling appropriate for a pet care business, rather than a cold or clinical look.

Navigation and structure

Every page shares the same header, navigation, and footer, so visitors always know where they are and how to get anywhere else on the site. Content is grouped into clearly bordered, rounded sections (hero banner, service cards, highlight box, FAQ list) to keep each page scannable.

Responsiveness

A media query at 700px switches the layout for smaller screens: the navigation links collapse behind a Menu button, and the three service cards stack vertically instead of sitting side by side, so the site remains usable on phones and tablets without a separate mobile site.

Accessibility

- All images include descriptive alt text (e.g., “Two dogs sitting together outside”).
- Form fields use matching label/for and id attributes so labels are properly associated with their inputs.
- Headings follow a logical order (`h1` for the site name, `h2` for section titles, `h3` for card titles) rather than skipping levels for visual effect.
- Interactive elements (menu button, FAQ questions, slideshow controls) are implemented as real `<button>` elements, which are keyboard-focusable by default.

5. Challenges and Solutions

Keeping four pages consistent

With four separate HTML files, there was a risk of the header, navigation, or footer drifting out of sync as changes were made. This was managed by keeping that markup identical across pages and relying on shared ids/classes, so a single `style.css` and `script.js` file could serve every page without page-specific code.

Building the slideshow without a page reload

Advancing the gallery needed to feel instant. The solution was to store all slideshow images and captions in a single JavaScript array of objects, track the current position with an index variable, and update the image, alt text, and caption in place — wrapping back to the first or last image when the index goes out of bounds.

Hiding FAQ answers without breaking accessibility

FAQ answers needed to start hidden and expand on click. Rather than hard-coding inline styles into the HTML, each answer starts hidden through a CSS rule, and JavaScript toggles the element's display property between "none" and "block" when its question is clicked.

Mobile navigation without a framework

Collapsing the navigation on small screens without a JavaScript framework was solved with a simple `classList.toggle("show")` call on the nav element, paired with a CSS rule that only displays the nav when the "show" class is present and the screen is narrow.

Staying within a four-week, front-end-only scope

The biggest risk was time. Keeping the JavaScript features intentionally simple — event listeners and DOM updates rather than any backend logic — kept the project achievable within the timeline while still meeting every requirement in the proposal.

6. Lessons Learned

- Planning a consistent header/nav/footer structure before writing any page-specific content saved significant time and prevented inconsistency across pages.
- Small, event-driven JavaScript (`addEventListener`, `classList.toggle`, simple array indexing) can create noticeably interactive features without needing a framework.
- Designing mobile-first, or at least testing the mobile breakpoint early, made the responsive CSS far easier than trying to retrofit it at the end.
- Accessibility basics (alt text, label associations, heading order) are inexpensive to build in from the start but easy to forget if left until the end of a project.
- Wireframing the four pages before writing code made the HTML structure for each page fast to build, since the layout decisions were already made.

7. Wireframes: Final Comparison

The final site follows the original wireframes closely, with no major structural changes:

- Home: header + nav, hero section with a call-to-action button, three service cards, a "why choose us" list, and footer — built exactly as planned.
- Services: header + nav, pricing table, and FAQ section — built exactly as planned.
- Gallery: header + nav and an image slideshow with Previous/Next controls — built exactly as planned.
- Contact: header + nav and a form collecting name, email, service interest, and message — built exactly as planned, with an added confirmation message area (`#formMessage`) that was implied but not explicitly drawn in the original wireframe.